

GIORNO 11

Web scraping: trasformare una pagina web in dati

Cosa saprete fare stasera

Estrarre i campi di un elenco da una pagina salvata, cercandoli dentro la scheda
Trasformare "1.249,00 €" in un numero, e scartare con il motivo ciò che non lo è
Seguire il link «Successiva» su più pagine, con un tetto e una pausa fra le letture
Decidere se uno scraper si può lanciare: robots.txt e una lista di domande
Salvare i record in CSV e in JSON, e rileggerli contando che ci siano tutti
Dire perché un'estrazione trova zero elementi, guardando la pagina
e non rilanciando il programma a tentativi

Oggi non imparate un linguaggio nuovo: imparate a leggere un formato nuovo
con tutto quello che avete già. Cicli, funzioni, dizionari, file, eccezioni.

1. RIPASSO E DOMANDE

Il lettore di file del Giorno 08, e il perimetro di oggi

1.1 Dal file alla struttura dati: oggi su una pagina

```
def carica_spedizioni(percorso):  
    spedizioni = []  
  
    # 1. read_text() legge tutto il file in una stringa (Giorno 08 $7.5),  
    #    splitlines() la spezza in righe senza lasciare i \n attaccati.  
    for riga in percorso.read_text(encoding="utf-8").splitlines():  
        codice, zona, peso = riga.split(";")  
  
        # 2. Il peso arriva come testo: si converte prima di conservarlo.  
        spedizioni.append((codice, zona, float(peso)))  
  
    return spedizioni  
  
PROVA = Path("spedizioni_prova.txt")  
  
PROVA.write_text("LS-01;NORD;12.5\nLS-02;SUD;3.0\nLS-03;ISOLE;7.25\n",  
                 encoding="utf-8")  
  
print(carica_spedizioni(PROVA))  
  
# 3. La pagina di oggi: anche lei e' un file di testo, con un'altra estensione.  
PAGINA = Path("dati") / "novastore" / "catalogo" / "pagina_1.html"  
print(PAGINA.is_file(), PAGINA.suffix)  
  
# TRABOCCHETTO: con .split("\n") al posto di splitlines() il file che finisce  
#    con un a capo produce una riga vuota in fondo. "".split(";") da' [''], e  
#    l'unpacking si ferma: ValueError: not enough values to unpack  
#    (expected 3, got 1).
```

1.2 Prerequisiti operativi: strip, get, try/except, json

```
import json

# 1. Giorno 03 $5.2: .strip() toglie spazi e a capo dai bordi, non dal mezzo.
print(repr(" Cuffie wireless\n".strip()))

# 2. Giorno 07 $11.1: la chiave che manca non ferma niente, esce il ripiego.
record = {"codice": "NS-2001", "nome": "Cuffie wireless"}
print(record.get("prezzo", "n.d."))

# 3. Giorno 09 $5.2: il testo che float() rifiuta diventa un messaggio.
for testo in ["49.90", "49,90"]:
    try:
        print(float(testo))
    except ValueError as e:
        print(f"[ERRORE] {e}")

# 4. Giorno 08 $12.2: il dizionario diventa testo JSON.
print(json.dumps({"codice": "NS-2001", "prezzo": 49.9}))

# TRABOCCHETTO: "49,90" e' il prezzo come lo scrive un sito italiano, e
# float() lo rifiuta. Oggi ogni prezzo arrivera' cosi', con in piu' il
# simbolo dell'euro: il capitolo 8 e' la cura.

# 'Cuffie wireless'

# n.d.

# 49.9

# [ERRORE] could not convert string to float: '49,90'

# {"codice": "NS-2001", "prezzo": 49.9}
```

1.3 Perimetro: le prime librerie esterne, niente rete

```
import json

from bs4 import BeautifulSoup

# cap. 3: la pagina e' testo con i tag. Qui e' in miniatura, una scheda sola.

HTML = """

<li class="prodotto" data-codice="NS-2001" data-categoria="AUDIO">

    <h2 class="nome">Cuffie wireless</h2>

    <span class="prezzo">49,90 &euro;</span>

</li>

"""

pagina = BeautifulSoup(HTML, "html.parser")          # cap. 4: l'albero
scheda = pagina.find("li", class_="prodotto")         # cap. 5: la scheda
testo_prezzo = scheda.find("span", class_="prezzo").get_text() # cap. 7
# cap. 8: dal testo al numero, con la virgola italiana del Giorno 03 $5.4.
prezzo = float(testo_prezzo.replace("&euro;", "").strip().replace(",", "."))
record = {"codice": scheda["data-codice"], "prezzo": prezzo} # cap. 7-8
print(json.dumps(record))                                # cap. 14

# TRABOCCHETTO: nel sorgente il simbolo e' scritto &euro;, ma .replace("&euro;")
# funziona lo stesso. Chi ha tradotto &euro; in € prima che arrivasse al
# vostro codice lo vedete al $3.4.

# {"codice": "NS-2001", "prezzo": 49.9}
```

2. INSTALLARE LE LIBRERIE: pip, beautifulsoup4, requests

2.1 Libreria standard e librerie esterne: perché serve pip

```
import sys

MODULI = ["json", "pathlib", "csv", "urllib", "time", "bs4", "requests"]

# 1. sys.stdlib_module_names e' l'elenco dei moduli della libreria
# standard, scritto dentro Python stesso. Con "in" si chiede se c'e'.
for nome in MODULI:
    if nome in sys.stdlib_module_names:
        print(f"{nome:<10}viene con Python")
    else:
        # 2. Tutto quello che non e' nell'elenco va installato a parte.
        print(f"{nome:<10}va installato")

# TRABOCCHETTO: l'elenco dice che cosa arriva CON Python, non che cosa c'e'
# sulla vostra macchina. Anche dove bs4 e' gia' installato, la riga dice
# "va installato": la risposta e' la stessa su ogni computer.

# json      viene con Python
# pathlib   viene con Python
# csv       viene con Python
# urllib     viene con Python
# time      viene con Python
# bs4       va installato
# requests  va installato
```


2.1 Che cosa fanno le due librerie: requests e bs4

Libreria	Che cosa riceve	Che cosa restituisce	Che cosa non fa
requests: client HTTP	un indirizzo, come str	la risposta del server: codice di stato, intestazioni, corpo come str	non interpreta l'HTML, non esegue JavaScript
bs4: parsing e interrogazione dell'HTML	il testo della pagina, come str, e il nome del parser	l'albero degli elementi, con i metodi per cercare tag, attributi, testo	non scarica, non conosce la rete, non esegue JavaScript

Il confine fra le due e' una stringa: requests la produce, BeautifulSoup la consuma. Per questo negli esercizi `read_text()` prende il posto di requests senza toccare il resto.

2.2 python -m pip install: installare per l'interprete giusto

```
# Nel TERMINALE, non nel file Python. In PyCharm il terminale e' Alt+F12.

# Windows:          python -m pip install beautifulsoup4 requests
# macOS e Linux:    python3 -m pip install beautifulsoup4 requests

# python -m pip: il pip del Python che esegue i vostri programmi.

# Finiscono nella site-packages di quell'interprete: la .venv del progetto.


# 1. Il nome giusto: il PACCHETTO si chiama beautifulsoup4, il MODULO che
#    si importa si chiama bs4. Sono due nomi per la stessa libreria.
from bs4 import BeautifulSoup

print("Import di bs4: [OK]")


# 2. Il nome del pacchetto scritto dopo import: Python cerca un modulo con
#    quel nome, non lo trova fra quelli installati, e si ferma.

# TRABOCCHETTO: pip ha appena detto "Successfully installed beautifulsoup4",
#    e l'import con quello stesso nome fallisce. Non e' l'installazione.
import beautifulsoup4


# Import di bs4: [OK]

# Traceback (most recent call last):

#   File "prova.py", line 11, in <module>
#       import beautifulsoup4

# ModuleNotFoundError: No module named 'beautifulsoup4'
```

2.3 Verificare l'installazione: import bs4 e sys.executable

```
import sys

from bs4 import BeautifulSoup


# 1. Una pagina di una riga, scritta dentro il programma: nessun file e
# nessuna rete. Se questo gira, bs4 e' installato per QUESTO Python.
prova = BeautifulSoup("<p>bs4 <b>funziona</b></p>", "html.parser")

# 2. Il testo del paragrafo con i tag tolti: <b> sparisce, le parole no.
print(prova.p.get_text())

# 3. Il nome del tipo dell'oggetto costruito: lo ritrovate al §4.2.
print(type(prova).__name__)

# TRABOCCHETTO: su un Python senza bs4 questo blocco non arriva alla riga
# del print: si ferma alla prima, con il traceback qui sotto.


# 4. Quale Python sta eseguendo questo file: il percorso del programma.
print(sys.executable)

# TRABOCCHETTO: se sys.executable nomina un percorso diverso da quello
# del pannello Python Interpreter di PyCharm, avete installato per un
# Python e state eseguendo con un altro. Il comando va rilanciato con
# il Python giusto davanti a -m pip.


# bs4 funziona

# BeautifulSoup
```

3. COM'È FATTA UNA PAGINA WEB: L'HTML

Visualizza sorgente pagina: la stessa pagina, vista da un programma

3.1 Una pagina è un file di testo: pagina_1.html

```
from pathlib import Path

# 1. Il percorso relativo parte dalla directory di lavoro (Giorno 08 §7.1):
#     gli esempi di oggi girano in una cartella che contiene dati/.
PERCORSO = Path("dati") / "novastore" / "catalogo" / "pagina_1.html"
testo_html = PERCORSO.read_text(encoding="utf-8")

# 2. Una pagina web letta dal disco è una stringa: Giorno 08 §2.2.
print(type(testo_html))

print("Contiene le cuffie:", "Cuffie wireless" in testo_html)

# 3. Un conteggio grezzo, con .count() del Giorno 02 §10.4.
print('Occorrenze di "<li":', testo_html.count("<li"))

# TRABOCCHETTO: senza encoding="utf-8" su Windows la lettura usa un'altra
#     codifica, e il "Novità" della pagina 2 diventa "NovitÃ ": lo stesso
#     guasto del Giorno 08 §2.4, su una pagina invece che su un registro.

# <class 'str'>
# Contiene le cuffie: True
# Occorrenze di "<li": 6
```

3.2 Anatomia di un tag: nome, attributi, contenuto, chiusura

```
<li class="prodotto" data-codice="NS-2001" data-categoria="AUDIO">
  <h2 class="nome">Cuffie wireless</h2>
  <span class="prezzo">49,90 &euro;</span>
</li>
```

*Un tag e' il segno fra parentesi angolari: <li ...> apre, chiude. Dopo il nome stanno gli attributi, coppie nome="valore". Fra apertura e chiusura il contenuto, e tutto insieme e' un elemento: questo li ne contiene altri due. <meta> e
 non si chiudono mai.*

3.2 Gli attributi sono un dizionario: il tag smontato a mano

```
# Il tag di apertura della scheda, copiato dalla pagina come testo.
APERTURA = '<li class="prodotto" data-codice="NS-2001" data-categoria="AUDIO">'

# 1. .strip() con un argomento toglie QUEI caratteri dai bordi: via < e >.
interno = APERTURA.strip("<>")

# 2. Il primo pezzo e' il NOME del tag, gli altri sono gli attributi.
pezzi = interno.split(" ")

nome_tag = pezzi[0]

attributi = {}

for pezzo in pezzi[1:]:

    # 3. chiave="valore": si taglia al primo uguale, si tolgono le virgolette.
    chiave, valore = pezzo.split("=", 1)

    attributi[chiave] = valore.strip('\'')

print("Nome del tag:", nome_tag)

print("Attributi:", attributi)

print("Categoria:", attributi["data-categoria"])

# TRABOCCHETTO: con class="prodotto esaurito" lo split(" ") spezza anche il
# valore, e il pezzo 'esaurito' non contiene un uguale. L'unpacking si
# ferma: ValueError: not enough values to unpack (expected 2, got 1).

# Nome del tag: li

# Attributi: {'class': 'prodotto', 'data-codice': 'NS-2001', 'data-categoria': 'AUDIO'}

# Categoria: AUDIO
```

3.3 Gli attributi di chi estrae: id, class, href, data-*

```
PERCORSO = Path("dati") / "novastore" / "catalogo" / "pagina_1.html"

testo_html = PERCORSO.read_text(encoding="utf-8")


# 1. id: unico. Lo spazio davanti evita di contare pezzi di altri nomi.
print("Attributi id:", testo_html.count(' id="'))

# 2. class: la stessa etichetta su tutte le schede. Due modi di cercarla.
print('class="prodotto"  :', testo_html.count('class="prodotto"'))
print('class="prodotto   :', testo_html.count('class="prodotto'))

# 3. data-*: il dato pulito che la pagina tiene per se'.
print('data-categoria="AUDIO":', testo_html.count('data-categoria="AUDIO"'))

# 4. href: uno per ogni link che porta da qualche parte.
print("Attributi href:", testo_html.count("href="))


# TRABOCCHETTO: con le virgolette chiuse il conteggio dice 5, non 6. La
#   scheda esaurita ha class="prodotto esaurito", e dopo prodotto viene uno
#   spazio, non le virgolette. Il testo non sa che sono due classi: il
#   parser si', e il $7.4 lo mostra.


# Attributi id: 1
# class="prodotto"  : 5
# class="prodotto   : 6
# data-categoria="AUDIO": 2
# Attributi href: 14
```


3.4 Le entità HTML: €, &, e che cosa diventano

```
from bs4 import BeautifulSoup

# Quattro pezzi di testo come li scrive il sorgente, entita' comprese.
SORGENTI = ["Bass &amp; Treble", "89,00 &euro;", "24,50&nbsp;&euro;",
            "&lt;br&gt;"]

for sorgente in SORGENTI:
    # 1. Il parser legge il pezzo e restituisce il testo gia' tradotto.
    testo = BeautifulSoup(sorgente, "html.parser").get_text()

    # 2. !r stampa la rappresentazione: rende visibile cio' che non si vede.
    print(f"{sorgente:<20} -> {testo!r}")

# TRABOCCHETTO: la terza riga non mostra uno spazio ma \xa0. E' lo spazio
# che non va a capo, e per Python non e' uno spazio qualunque: il
# riquadro qui sotto mostra dove fa danni.

# Bass &amp; Treble      -> 'Bass & Treble'
# 89,00 &euro;          -> '89,00 €'
# 24,50&nbsp;&euro;        -> '24,50\xa0€'
# &lt;br&gt;              -> '<br>'
```

3.5 Cercare con str.find() e lo slicing: dove smette

```
# testo_html: pagina_1.html letta con read_text(), come al §3.1.

MARCA = 'class="prezzo">'

# 1. Il primo prezzo della pagina: si trova la marca e si taglia fino al <.

inizio = testo_html.find(MARCA) + len(MARCA)

fine = testo_html.find("<", inizio)

print("Primo prezzo:", testo_html[inizio:fine])

# 2. Per avere quello delle cuffie si parte dal loro codice.

dopo_codice = testo_html.find("NS-2001")

inizio = testo_html.find(MARCA, dopo_codice) + len(MARCA)

fine = testo_html.find("<", inizio)

print("Cuffie:", testo_html[inizio:fine])

# 3. Lo stesso span con un attributo in piu', scritto dopo class.

VARIANTE = '<span class="prezzo" title="IVA inclusa">49,90 &euro;</span>'

inizio = VARIANTE.find(MARCA) + len(MARCA)

fine = VARIANTE.find("<", inizio)

print("Variante:", VARIANTE[inizio:fine])

# TRABOCCHETTO: find() che non trova restituisce -1, non un errore

# (Giorno 02 §10.4). -1 + 15 fa 14, lo slicing parte dal quattordicesimo

# carattere, ed esce spazzatura senza nessun traceback.


# Primo prezzo: 15,90 &euro;

# Cuffie: 49,90 &euro;

# Variante: rezzo" title="IVA inclusa">49,90 &euro;
```

4. L'ALBERO DEL DOCUMENTO

Non una lista di righe: una struttura a livelli, uno dentro l'altro

4.1 Dal testo all'albero: genitore, figlio, discendente

```
html
├── head
│   └── title "NovaStore - Catalogo, pagina 1 di 3"
└── body
    ├── header#testata
    └── main
        ├── h1 "Catalogo prodotti"
        ├── aside.offerta-giorno
        │   └── p
        │       └── span.prezzo "15,90 €"
        ├── ul.catalogo
        │   ├── li.prodotto [NS-2001]
        │   │   ├── h2.nome "Cuffie wireless"
        │   │   ├── span.prezzo "49,90 €"
        │   │   ├── span.disponibilita "Disponibile"
        │   │   └── a.scheda "Vedi scheda"
        │   ├── li.prodotto [NS-2002]
        │   └── ... altre quattro schede
        └── nav.paginazione
```

Il DOM, Document Object Model: la pagina come albero, ogni punto un nodo. Per li.prodotto [NS-2001]: genitore ul.catalogo, figli h2, span e a, fratello NS-2002. Discendenti: figli, figli dei figli e così via. Un figlio è sempre un discendente, non il contrario.

4.1 Un albero costruito a mano: tuple e cicli annidati

```
# Ogni nodo e' una tupla (nome, testo, figli): Giorno 07 $13.1.
SCHEDA = ("li", "", [
    ("h2", "Cuffie wireless", []),
    ("span", "49,90 €", []),
    ("a", "Vedi scheda", []),
])

CATALOGO = ("ul", "", [SCHEDA])

# 1. L'unpacking a tre nomi del Giorno 07 $13.3, sulla radice.
nome_ul, testo_ul, figli_ul = CATALOGO

discendenti = 0

# 2. Due livelli, due cicli annidati: il ciclo interno scorre i figli del li.
for nome_li, testo_li, figli_li in figli_ul:
    discendenti += 1

    print(f"{nome_ul} -> {nome_li} (figlio diretto)")

    for nome_foglia, testo_foglia, figli_foglia in figli_li:
        discendenti += 1

        print(f"{nome_ul} -> {nome_li} -> {nome_foglia} {testo_foglia}")

print(f"Figli di {nome_ul}: {len(figli_ul)}, discendenti: {discendenti}")

# TRABOCCHETTO: len(figli_ul) dice 1, non 4. Conta solo il primo livello:
#   h2, span e a sono discendenti di ul, ma non suoi figli.

# ul -> li (figlio diretto)
# ul -> li -> h2 Cuffie wireless
# ul -> li -> span 49,90 €
# ul -> li -> a Vedi scheda
# Figli di ul: 1, discendenti: 4
```

4.2 BeautifulSoup(testo, "html.parser"): costruire l'albero

```
PERCORSO = Path("dati") / "novastore" / "catalogo" / "pagina_1.html"

testo_html = PERCORSO.read_text(encoding="utf-8")

# 1. La riga del capitolo: dalla stringa all'albero. Il secondo argomento
#     sceglie il parser, e va scritto sempre.
pagina = BeautifulSoup(testo_html, "html.parser")

# 2. Prima e dopo: stesso contenuto, due tipi diversi (Giorno 02 $5.1).
print(type(testo_html))
print(type(pagina))

# 3. Nell'albero si entra per nome: .title e' il primo <title> della pagina.
print(pagina.title.get_text())
print(pagina.h1.get_text())
print(type(pagina.title))

# TRABOCCHETTO: print(pagina) non mostra un albero disegnato. Ristampa
#     tutto l'HTML, ottanta righe: l'albero sa risciversi come testo, ma la
#     sua forma si esplora con i metodi, non con print.

# <class 'str'>
# <class 'bs4.BeautifulSoup'>
# NovaStore - Catalogo, pagina 1 di 3
# Catalogo prodotti
# <class 'bs4.element.Tag'>
```

4.3 Navigare per nome: pagina.title, .parent, .name

```
# HTML: due <li class="prodotto"> dentro <ul class="catalogo">, a capo e rientrati.

pagina = BeautifulSoup(HTML, "html.parser")

# 1. Il primo <span>, poi su di un livello alla volta con .parent.

prezzo = pagina.span

scheda = prezzo.parent

print(prezzo.name, "->", scheda.name, "->", scheda.parent.name,

      "->", scheda.parent.parent.name)

print(scheda.h2.get_text())

ul = pagina.ul

figli = list(ul.children)

quanti_li = 0

for figlio in figli:

    if figlio.name == "li":

        quanti_li += 1

print("Figli di <ul>:", len(figli), "- di cui <li>:", quanti_li)

print("Il primo figlio:", repr(str(figli[0])))

# TRABOCCHETTO: len(list(ul.children)) dice 5, non 2. Fra un tag e l'altro

# ci sono l'a capo e i rientri dell'HTML, e anche quelli sono figli: nodi

# di solo testo. BeautifulSoup li riduce a un solo '\n', rientro perso.


# span -> li -> ul -> [document]

# Cuffie wireless

# Figli di <ul>: 5 - di cui <li>: 2

# Il primo figlio: '\n'
```

4.4 «Ispeziona» nel browser: il selettore copiato si accorcia

```
# 1. Come lo copia il browser: il percorso intero, con la posizione (3).
LUNGO = "body > main > ul > li:nth-child(3) > span.prezzo"

# 2. Accorciato a mano: la scheda si riconosce dal codice, non dal posto.
CORTO = 'li[data-codice="NS-2003"] span.prezzo'

pagina = BeautifulSoup(testo_html, "html.parser")
print("Prima:", pagina.select_one(LUNGO).get_text(), "|",
      pagina.select_one(CORTO).get_text())

# 3. Il sito aggiunge un prodotto in cima all'elenco. Nient'altro cambia.
NUOVA = ('<li class="prodotto" data-codice="NS-2000">'
        '<span class="prezzo">5,00 &euro;</span></li>')
modificato = testo_html.replace('<ul class="catalogo">',
                                '<ul class="catalogo">' + NUOVA)

pagina = BeautifulSoup(modificato, "html.parser")
print("Dopo: ", pagina.select_one(LUNGO).get_text(), "|",
      pagina.select_one(CORTO).get_text())

# TRABOCCHETTO: dopo la modifica il selettore lungo restituisce il prezzo
# della tastiera, 79,00, senza nessun errore. La terza scheda adesso e'
# un'altra, e il programma non ha modo di accorgersene.

# Prima: 1.249,00 € | 1.249,00 €
# Dopo: 79,00 € | 1.249,00 €
```


5. TROVARE GLI ELEMENTI: find() E find_all()

Il primo, oppure tutti: e che cosa restituiscono quando non trovano niente

5.1 find(): il primo elemento che corrisponde, oppure None

```
PERCORSO = Path("dati") / "novastore" / "catalogo" / "pagina_1.html"

pagina = BeautifulSoup(PERCORSO.read_text(encoding="utf-8"), "html.parser")

# 1. Il primo <h1>: ce n'e' uno solo, e il risultato e' quello atteso.
print(pagina.find("h1").get_text())

# 2. Il primo <a> in ordine di documento: il logo in testata.
print(pagina.find("a"))

# 3. Il primo <span>: il prezzo dell'offerta del giorno, fuori dal catalogo.
print(pagina.find("span"))

# 4. Un tag che nella pagina non c'e': nessun errore, torna None.
risultato = pagina.find("table")
print(risultato, type(risultato))

# TRABOCCHETTO: find("span") non restituisce il prezzo delle cuffie ma
# quello dell'offerta, 15,90, perche' l'offerta sta piu' in alto nel
# sorgente. find() non sa quale volevate: sa solo quale viene prima.

# Catalogo prodotti
# <a class="logo" href="/">NovaStore</a>
# <span class="prezzo">15,90 €</span>
# None <class 'NoneType'>
```

5.2 find_all(): tutti gli elementi, in una lista da scorrere

```
PERCORSO = Path("dati") / "novastore" / "catalogo" / "pagina_1.html"

pagina = BeautifulSoup(PERCORSO.read_text(encoding="utf-8"), "html.parser")

# 1. Tutti i <li>: un oggetto di tipo ResultSet, che e' una lista.
schede = pagina.find_all("li")

print(type(schede).__name__, "con", len(schede), "elementi")

# 2. Si scorre come una lista, con il contatore che parte da 1.
for numero, scheda in enumerate(schede, 1):

    print(f"{numero}. {scheda.h2.get_text()}")

# 3. Nessun elemento trovato: la lista e' vuota, e non e' None.
tabelle = pagina.find_all("table")

print("Tabelle:", len(tabelle), tabelle)

# TRABOCCHETTO: schede.get_text() non esiste. get_text() e' di un singolo
# elemento, e schede e' una lista: si ferma con AttributeError, e bs4
# aggiunge al messaggio una domanda: "Did you call find_all() when you
# meant to call find()?"

# ResultSet con 6 elementi

# 1. Cuffie wireless
# 2. Tastiera meccanica
# 3. Monitor 4K 32 pollici
# 4. Cavo USB-C 2 m
# 5. Casse bluetooth
# 6. Mouse wireless

# Tabelle: 0 []
```

5.3 Cercare per classe e per attributo: `class_` e `attrs={...}`

```
# pagina: pagina_1.html, come al §5.1.

# 1. Per classe: trova anche la scheda con class="prodotto esaurito".
prodotti = pagina.find_all("li", class_="prodotto")
print("class_ prodotto:", len(prodotti))

# 2. La scheda esaurita ha DUE classi: basta nominarne una per trovarla.
esauriti = pagina.find_all("li", class_="esaurito")
print("class_ esaurito:", len(esauriti), "->", esauriti[0].h2.get_text())

# 3. Il trattino non sta in un nome di argomento: dizionario (Giorno 07).
audio = pagina.find_all("li", attrs={"data-categoria": "AUDIO"})
print("data-categoria AUDIO:", len(audio))

for scheda in audio:
    print("  ", scheda.h2.get_text())

# TRABOCCHETTO: find_all("li", class="prodotto") senza trattino basso non
# parte nemmeno. SyntaxError alla lettura del file, prima della prima
# riga: nessun print compare, neanche quelli sopra.

# class_ prodotto: 6
# class_ esaurito: 1 -> Casse bluetooth
# data-categoria AUDIO: 2
#   Cuffie wireless
#   Casse bluetooth
```

5.4 Cercare dentro la scheda, non dentro la pagina

```
# pagina: pagina_1.html, come al $5.1.

schede = pagina.find_all("li", class_="prodotto")
prezzi = pagina.find_all("span", class_="prezzo")
print(f"Prezzi nella pagina: {len(prezzi)}, schede: {len(schede)}")


# 1. SBAGLIATO: due liste parallele, legate soltanto dall'indice.
print("-- liste parallele --")

for indice, scheda in enumerate(schede):

    print(f"{scheda.h2.get_text():<24}{prezzi[indice].get_text()}")


# 2. GIUSTO: il prezzo si cerca DENTRO la scheda, con find() chiamato
# sulla scheda e non sulla pagina. Un record alla volta.
print("-- dentro la scheda --")

for scheda in schede:

    prezzo = scheda.find("span", class_="prezzo")

    print(f"{scheda.h2.get_text():<24}{prezzo.get_text()}")


# TRABOCCHETTO: la versione sbagliata non dà nessun errore. Stampa sei
# righe ben incolonnate, e ogni prezzo è quello del prodotto di sopra.
# Il Monitor 4K costa 79,00 e il cavo USB 1.249,00.
```

5.4 Sei righe pulite, sei prezzi sbagliati

Prezzi nella pagina: 7, schede: 6

-- liste parallele --

Cuffie wireless	15,90 €
Tastiera meccanica	49,90 €
Monitor 4K 32 pollici	79,00 €
Cavo USB-C 2 m	1.249,00 €
Casse bluetooth	9,90 €
Mouse wireless	59,00 €

-- dentro la scheda --

Cuffie wireless	49,90 €
Tastiera meccanica	79,00 €
Monitor 4K 32 pollici	1.249,00 €
Cavo USB-C 2 m	9,90 €
Casse bluetooth	59,00 €
Mouse wireless	19,90 €

La regola, sempre: si trovano le schede, si scorrono con un for, e dentro il ciclo ogni campo si cerca con scheda.find(...), che guarda solo fra i discendenti della scheda. E' il modo di dire il prezzo di QUESTO prodotto invece di un prezzo.

5.5 Il None di find() è il None del Giorno 06: AttributeError

```
# Tre schede di pagina_2.html in miniatura: la seconda non ha il prezzo.

HTML = """

<li class="prodotto" data-codice="NS-2007"><h2 class="nome">Webcam HD</h2>

    <span class="prezzo">49,00 &euro;</span></li>

<li class="prodotto" data-codice="NS-2010"><h2 class="nome">Docking station</h2>

    <span class="disponibilita">Disponibile</span></li>

<li class="prodotto" data-codice="NS-2011"><h2 class="nome">Proiettore portatile</h2>

    <span class="prezzo">su richiesta</span></li>

"""

pagina = BeautifulSoup(HTML, "html.parser")

for scheda in pagina.find_all("li", class_="prodotto"):

    nome = scheda.find("h2", class_="nome").get_text()

    # 1. Sulla Docking station find() non trova lo span: restituisce None.

    # 2. None non ha metodi, e .get_text() chiesto a None ferma tutto.

    prezzo = scheda.find("span", class_="prezzo").get_text()

    print(f"{nome:<22}{prezzo}")

# TRABOCCHETTO: il traceback parla di NoneType, che nella riga non compare:

#   il None l'ha prodotto find(), un attimo prima, nella stessa riga.


# Webcam HD           49,00 €

# Traceback (most recent call last):

#   File "prova.py", line 18, in <module>

#       prezzo = scheda.find("span", class_="prezzo").get_text()

# AttributeError: 'NoneType' object has no attribute 'get_text'
```

5.5 testo_campo(scheda, nome_tag, classe): il None controllato

```
def testo_campo(scheda, nome_tag, classe):

    """Restituisce il testo pulito del campo, oppure None se il campo manca."""

    elemento = scheda.find(nome_tag, class_=classe)

    # 1. Il controllo PRIMA dell'uso: la guardia del Giorno 04, su un tag.

    if elemento is None:

        return None

    return elemento.get_text(" ", strip=True)


# HTML: le due schede difficili della pagina 2, prezzo assente e a parole.
pagina = BeautifulSoup(HTML, "html.parser")

for scheda in pagina.find_all("li", class_="prodotto"):

    nome = testo_campo(scheda, "h2", "nome")

    prezzo = testo_campo(scheda, "span", "prezzo")

    if prezzo is None:

        print(f"{nome:<22} [!] prezzo assente")

    else:

        print(f"{nome:<22}{prezzo}")


# TRABOCCHETTO: "su richiesta" passa il controllo, perche' non e' None. Il
# campo c'e', e' il suo contenuto a non essere un numero: quello e' un
# altro problema, e lo risolve prezzo_da_testo() al capitolo 8.


# Docking station      [!] prezzo assente

# Proiettore portatile su richiesta
```


6. I SELETTORI CSS: `select()` E `select_one()`

La stessa lingua del browser e di chi costruisce il sito

6.1 Tag, classe, id: li.prodotto, #tariffe, .nome

```
# pagina: pagina_1.html, come al $5.1.

# 1. Quattro selettori in una lista, provati uno alla volta con select().
SELETTORI = ["li.prodotto", ".nome", "li.esaurito", "prodotto"]

for selettore in SELETTORI:

    trovati = pagina.select(selettore)

    print(f"{selettore:<13}-> {len(trovati)} elementi")

# 2. select_one() con l'id: un elemento, non una lista. Nel catalogo

#    l'unico id e' quello della testata; #tariffe sta nella pagina LogiSud.
testata = pagina.select_one("#testata")

print(f"{'#testata':<13}-> <{testata.name}>")

# 3. Nessun elemento: select_one() da' None, come find() al $5.1.

print(f"{'table':<13}-> {pagina.select_one('table')}")


# TRABOCCHETTO: "prodotto" senza il punto cerca un TAG che si chiama
#    <prodotto>, che non esiste: zero elementi, e nessun errore. Un punto
#    mancante e' il selettore sbagliato piu' frequente della giornata.


# li.prodotto  -> 6 elementi
# .nome        -> 6 elementi
# li.esaurito  -> 1 elementi
# prodotto     -> 0 elementi
# #testata     -> <header>
# table        -> None
```

6.2 Discendente e figlio diretto: lo spazio contro il >

```
HTML = """
<ul class="catalogo">

  <li class="prodotto">Kit videoconferenza

    <ul class="contenuto">

      <li>Webcam HD</li>

      <li>Cuffie wireless</li>

    </ul>

  </li>

  <li class="prodotto">Mouse wireless</li>
</ul>
"""

pagina = BeautifulSoup(HTML, "html.parser")
tutti = pagina.select("ul.catalogo li")
diretti = pagina.select("ul.catalogo > li")
print("Con lo spazio (discendenti):", len(tutti))
for elemento in tutti:
    print("  ", elemento.get_text(" ", strip=True))
print("Con il > (figli diretti):", len(diretti))
for elemento in diretti:
    print("  ", elemento.get_text(" ", strip=True))

# TRABOCCHETTO: il > sceglie QUALI elementi prendere, non QUANTO testo
# portano con se'. Il kit, anche preso come figlio diretto, contiene il
# testo dei suoi pezzi: get_text() raccoglie tutti i discendenti.
```

6.3 Selettori per attributo: [data-codice], ^= e \$=

```
PERCORSO = Path("dati") / "villanova" / "albo.html"

pagina = BeautifulSoup(PERCORSO.read_text(encoding="utf-8"), "html.parser")

# 1. Una lista di tuple (etichetta, selettore), come le tabelle del
#     Giorno 07 $14.1, scorsa con l'unpacking a due nomi.
CONTEGGI = [

    ("tag <a> in tutto", "a"),

    ("con href", "a[href]"),

    ("href ^= https", 'a[href^="https"]'),

    ("href $= .pdf", 'a[href$=".pdf"]'),

    ("href ^= mailto:", 'a[href^="mailto:"]'),

]

for etichetta, selettore in CONTEGGI:

    print(f"{etichetta:<18}{len(pagina.select(selettore)):>3}")

# 2. Gli indirizzi, letti con le quadre sull'elemento ($7.3).

print("Gli atti in PDF:")

for link in pagina.select('a[href$=".pdf"]'):

    print("  ", link["href"])

print("Gli indirizzi che cominciano con https:")

for link in pagina.select('a[href^="https"]'):

    print("  ", link["href"])

# TRABOCCHETTO: "comincia con https" non vuol dire "porta fuori dal sito".

#     Il secondo indirizzo dell'elenco e' quello del Comune stesso, scritto

#     per intero. Il criterio giusto per dire esterno e' il dominio ($9.6).
```

6.4 find o select: la tabella di decisione

```
# 1. Con find: prima le schede, poi il nome dentro ciascuna ($5.4).
con_find = []

for scheda in pagina.find_all("li", class_="prodotto"):
    con_find.append(scheda.find("h2", class_="nome").get_text())

# 2. Con select: un selettore solo, che scende dalla scheda al nome.
con_select = []

for nome in pagina.select("li.prodotto h2.nome"):
    con_select.append(nome.get_text())

# 3. Due liste di stringhe: == le confronta elemento per elemento.
print(con_find == con_select)

print(len(con_select), "nomi, dal primo all'ultimo:")

print(con_select[0], "...", con_select[-1])


# TRABOCCHETTO: select("h2.nome") senza la scheda davanti da' lo stesso
# risultato su QUESTA pagina, e un risultato diverso il giorno in cui il
# sito mette un h2.nome anche nella barra laterale.


# True

# 6 nomi, dal primo all'ultimo:

# Cuffie wireless ... Mouse wireless
```

7. ESTRARRE TESTO E ATTRIBUTI

Il testo che si vede e i dati negli attributi: due modi di sbagliare in silenzio

7.1 get_text(strip=True): senza tag e senza spazi di bordo

```
# L'HTML come lo scrive un sito vero: il nome va a capo, rientrato.

HTML = """<li class="prodotto">

    <h2 class="nome">

        Cuffie wireless

    </h2>

</li>"""

nome = BeautifulSoup(HTML, "html.parser").find("h2", class_="nome")

# 1. get_text() nudo: il testo con gli a capo e i rientri del sorgente.
print(repr(nome.get_text()))

# 2. strip=True pulisce i bordi, dentro bs4.
print(repr(nome.get_text(strip=True)))

# 3. Lo stesso risultato con il .strip() del Giorno 03, fuori da bs4.
print(repr(nome.get_text().strip()))

# 4. Il confronto con il testo nudo: risponde di no.

print(nome.get_text() == "Cuffie wireless")


# TRABOCCHETTO: un filtro scritto come if nome.get_text() == "Cuffie
#   wireless" non scatta mai, e non dà errori. È il \n rimasto attaccato
#   alla riga del Giorno 08 E2, arrivato da una pagina invece che da un file.

# '\n        Cuffie wireless\n    '
# 'Cuffie wireless'
# 'Cuffie wireless'

# False
```

7.2 Il separatore di get_text(): le parole che si incollano

```
HTML = '<h2 class="nome">Lampada da scrivania <em class="etichetta">Novità</em></h2>'
nome = BeautifulSoup(HTML, "html.parser").h2
```

```
# 1. Senza argomenti: i pezzi incollati come sono, spazio di coda compreso.
```

```
print(repr(nome.get_text()))
```

```
# 2. strip=True pulisce OGNI pezzo prima di incollarlo: lo spazio che
```

```
# stava in coda al primo pezzo sparisce, e le parole si toccano.
```

```
print(repr(nome.get_text(strip=True)))
```

```
# 3. Con un separatore visibile si vede quanti pezzi ci sono: due.
```

```
print(repr(nome.get_text("|", strip=True)))
```

```
# TRABOCCHETTO: "Lampada da scrivaniaNovità" finisce nel CSV del cap. 14
```

```
# senza errori, e il giorno dopo la ricerca per "scrivania" nel foglio di
```

```
# Giulia non trova la lampada.
```

```
# 'Lampada da scrivania Novità'
```

```
# 'Lampada da scrivaniaNovità'
```

```
# 'Lampada da scrivania|Novità'
```


7.3 Gli attributi: tag["href"] contro tag.get("href")

```
# pagina: l'albo di Villanova, dati/villanova/albo.html

# 1. Il primo link di un atto: l'attributo c'e', le quadre lo leggono.
primo_atto = pagina.select_one("a.atto")
print(primo_atto["href"])

# 2. Il link dell'archivio non ha href: .get() risponde None, o il ripiego.
disattivo = pagina.select_one("a.disattivo")
print(disattivo.get("href"))

print(disattivo.get("href", "(nessun indirizzo)"))

listino = {"NORD": 8.5, "SUD": 9.5}
print(listino.get("ESTERO", "(su preventivo)"))

# TRABOCCHETTO: le quadre su un attributo che manca si fermano come le
# quadre su una chiave che manca: KeyError con il nome fra apici.
print(disattivo["href"])

# atti/2026-0142.pdf
# None
# (nessun indirizzo)
# (su preventivo)
# Traceback (most recent call last):
#   File "prova.py", line 19, in <module>
#     print(disattivo["href"])
#   ...
# KeyError: 'href'
```

7.4 class è una lista: ['prodotto', 'esaurito']

```
# pagina: pagina_1.html, come al $5.1.
esaurita = pagina.select_one("li.esaurito")

# 1. class e' una lista, data-codice una stringa: dipende dall'attributo.
print(esaurita["class"])
print(esaurita["data-codice"])

# 2. Tre domande sulla stessa lista, e solo una e' quella giusta.
print(esaurita["class"] == "esaurito")
print("esaurito" in esaurita["class"])
print("esaur" in esaurita["class"])

# 3. Il campo disponibile di ogni scheda: vero se NON e' esaurita.
for scheda in pagina.find_all("li", class_="prodotto"):
    disponibile = "esaurito" not in scheda["class"]
    print(scheda["data-codice"], disponibile)

# TRABOCCHETTO: esaurita["class"] == "esaurito" confronta una lista con
# una stringa, e risponde False senza errori, su ogni scheda. Un filtro
# scritto cosi' dichiara disponibile tutto il catalogo.

# ['prodotto', 'esaurito']
# NS-2005
# False
# True
# False
```

8. DAL TESTO AL NUMERO: PULIRE I DATI ESTRATTI

`get_text()` restituisce sempre una stringa, come `input()` e come una riga di file

8.1 Il prezzo è una stringa: da "1.249,00 €" a 1249.0

```
# I testi dei prezzi come li restituisce testo_campo() ($5.5) sul catalogo.
TESTI = ["49,90 €", "1.249,00 €", "€ 29,00", "24,50\xa0€"]

print(f'passo 1':<10}{passo 2':<10}{passo 3':<10}{prezzo':>9}")

for testo in TESTI:

    # Passo 1: via il simbolo, poi gli spazi di bordo (Giorno 03 $5.2).

    # .strip() toglie anche il \xa0 di &nbsp; per Python e' uno spazio.
    passo_1 = testo.replace("€", "").strip()

    # Passo 2: via il punto delle migliaia, finché la virgola e' virgola.
    passo_2 = passo_1.replace(".", "")

    # Passo 3: la virgola italiana diventa il punto (Giorno 03 $5.4).
    passo_3 = passo_2.replace(",", ".")

    prezzo = float(passo_3)

    print(f'{passo_1:<10}{passo_2:<10}{passo_3:<10}{prezzo:>9.2f} EUR")

# TRABOCCHETTO: invertite i passi 2 e 3. "1.249,00" diventa "1.249.00" e poi
# "124900": float() lo accetta e il monitor costa 124900.00 EUR, cento
# volte tanto, senza errori. Saltate il passo 2, invece, e float() si ferma
# su '1.249.00' (E7): dei due sbagli, il traceback e' quello fortunato.

# passo 1    passo 2    passo 3    prezzo
# 49,90      49,90      49.90      49.90 EUR
# 1.249,00   1249,00   1249.00   1249.00 EUR
# 29,00      29,00      29.00      29.00 EUR
# 24,50      24,50      24.50      24.50 EUR
```

8.2 prezzo_da_testo(): converte, o solleva ValueError con il motivo

```
def prezzo_da_testo(testo):  
    """Converte il testo di un prezzo in float, oppure solleva ValueError.  
  
    Riceve cio' che restituisce testo_campo() ($5.5): una stringa o None.  
    Il messaggio comincia sempre con il motivo; il valore, se c'e', dopo ":".  
  
    """  
    # Caso 1: il campo non c'era. None va controllato PER PRIMO.  
    if testo is None:  
        raise ValueError("prezzo assente")  
    pulito = testo.replace("€", "").strip()  
    # Caso 2: il campo c'era, ma vuoto (o fatto di soli spazi).  
    if pulito == "":  
        raise ValueError("prezzo vuoto")  
    try:  
        return float(pulito.replace(".", "").replace(",", "."))  
    except ValueError as e:  
        # Caso 3: c'e' del testo e non e' un numero. L'errore tecnico diventa  
        # uno che dice motivo e valore; "from e" lo dichiara (Giorno 09 $11.5).  
        raise ValueError(f"prezzo non numerico: '{testo}'") from e  
  
# TRABOCCHETTO: spostate il controllo del None sotto la riga di .replace().  
# Con la Docking station, che il prezzo non ce l'ha, il programma si ferma  
# con AttributeError: 'NoneType' object has no attribute 'replace' - e  
# l'except ValueError del chiamante non lo raccoglie: e' un altro tipo.
```

8.3 Scartare e contare: il try dentro il for

```
# testo_campo() ($5.5) e prezzo_da_testo() ($8.2) stanno in cima al file.

percorso = Path("dati") / "novastore" / "catalogo" / "pagina_2.html"
pagina = BeautifulSoup(percorso.read_text(encoding="utf-8"), "html.parser")
schede = pagina.find_all("li", class_="prodotto")

validi = 0
totale = 0.0
scarti_per_motivo = {}

for scheda in schede:

    codice = scheda["data-codice"]

    try:

        prezzo = prezzo_da_testo(testo_campo(scheda, "span", "prezzo"))

    except ValueError as e:

        # Il motivo e' la parte prima dei due punti: split del Giorno 02 $10.3.
        motivo = str(e).split(":")[0]

        scarti_per_motivo[motivo] = scarti_per_motivo.get(motivo, 0) + 1

        print(f"  [!] {codice} {e}")

        continue

    validi += 1
    totale += prezzo

    print(f"  {codice} {prezzo:>7.2f} EUR")

# TRABOCCHETTO: togliete il continue. La scheda scartata arriva lo stesso a
# validi += 1, e prezzo vale ancora quello della scheda PRIMA (89.00, le
# cuffie). Risultato: "valide: 6, scartate: 0" e un totale di 367.50 EUR.
# Nessun errore: il Giorno 09 $9.6, identico, su una pagina web.
```

8.4 Il record come dizionario: cinque chiavi fisse

```
# CARTELLA = dati/novastore/catalogo; testo_campo() e prezzo_da_testo() in cima.

record = []

codici_visti = []

for nome_file in ["pagina_1.html", "pagina_3.html"]:

    testo_html = (CARTELLA / nome_file).read_text(encoding="utf-8")

    pagina = BeautifulSoup(testo_html, "html.parser")

    for scheda in pagina.find_all("li", class_="prodotto"):

        codice = scheda["data-codice"]

        # Il doppione si controlla PRIMA di estrarre: non costa niente.

        if codice in codici_visti:

            print(f"[!] {codice} già letto: doppione in {nome_file}")

            continue

        codici_visti.append(codice)

    try:

        prezzo = prezzo_da_testo(testo_campo(scheda, "span", "prezzo"))

    except ValueError as e:

        print(f"[!] {codice} scartato: {e}")

        continue

    # Un record, cinque chiavi fisse. data-categoria con .get() ($7.3).

    record.append({

        "codice": codice,

        "nome": testo_campo(scheda, "h2", "nome"),

        "categoria": scheda.get("data-categoria"),

        "prezzo": prezzo,

        "disponibile": "esaurito" not in scheda["class"],

    })
```

8.4 Il quinto record, e il contratto delle chiavi

```
print("Record:", len(record))

for chiave, valore in record[4].items():

    print(f"    {chiave:<12}{valore}")


# TRABOCCHETTO: scrivete "Prezzo" con la maiuscola in un record solo. Qui non
# succede niente; al $14.2 il salvataggio in CSV si ferma con ValueError:
# dict contains fields not in fieldnames: 'Prezzo'. Le chiavi sono un
# contratto fra chi estrae e chi salva, come i nomi dei campi del Giorno 08.


# [!] NS-2014 scartato: prezzo vuoto

# [!] NS-2006 già letto: doppiante in pagina_3.html

# Record: 9

#   codice      NS-2005
#   nome        Casse bluetooth
#   categoria    AUDIO
#   prezzo       59.0
#   disponibile False
```


8.5 Tre controlli dopo ogni estrazione

```
lette = 4

scartate = 1

record = [

    {"codice": "NS-2007", "nome": "Webcam HD", "categoria": "VIDEO", "prezzo": 49.0},

    {"codice": "NS-2008", "nome": "Supporto per portatile", "categoria": None,

    "prezzo": 24.5},

    {"codice": "NS-2012", "nome": None, "categoria": "ACCESSORI", "prezzo": 27.0},

]

# Controllo 1 - quanti. L'invariante: nessuna scheda persa per strada.
assert lette == len(record) + scartate, "schede perse per strada"

print(f"Lette: {lette} valide: {len(record)} scartate: {scartate}")

# Controllo 2 - il primo record, a occhio: i campi sono al posto giusto?
print("Primo:", record[0]["codice"], record[0]["nome"], record[0]["prezzo"])

# Controllo 3 - i None per campo: il conta-occorrenze del Giorno 07 $12.2.

vuoti = {}

for voce in record:

    for campo, valore in voce.items():

        if valore is None:

            vuoti[campo] = vuoti.get(campo, 0) + 1

print("Campi vuoti:", vuoti)

# TRABOCCHETTO: le due schede incomplete sono valide per il prezzo, quindi i

# controlli 1 e 2 passano. Senza il 3 arriverebbero al cruscotto con un None

# dentro, e il None esploderebbe alla prima .upper() con AttributeError:

# 'NoneType' object has no attribute 'upper' - il Giorno 06 $6.2.
```

9. TABELLE E LINK

Una lista di tuple e un conta-occorrenze: cose che conoscete

9.1 Una tabella HTML: tr, th, td, e la lista di tuple

```
# HTML: la tabella tariffe in miniatura, thead + tbody NORD/SUD/ESTERO + tfoot.

pagina = BeautifulSoup(HTML, "html.parser")

righe = []

for tr in pagina.find_all("tr"):

    celle = tr.find_all("td")

    # L'intestazione ha celle th, quindi zero td; il piede ne ha una sola.

    # Si controlla il numero di campi PRIMA dell'unpacking (Giorno 08 $9.2).

    if len(celle) != 3:

        print(f"[!] riga saltata: {len(celle)} celle td")

        continue

    testi = []

    for cella in celle:

        testi.append(cella.get_text(strip=True))

    righe.append(tuple(testi))

print("Righe di dati:", len(righe))

# Unpacking in testa al for: tanti nomi quanti campi (Giorno 07 $13.3).

for zona, prezzo_testo, consegna in righe:

    print(f"{zona:<8}{consegna:<22}{prezzo_testo.replace('€', '').strip()}")

# TRABOCCHETTO: senza il controllo sul numero di celle, l'unpacking della

# prima riga si ferma con ValueError: not enough values to unpack

# (expected 3, got 0). E se la saltate a mano, il piede fa lo stesso con

# "got 1": la riga corta del Giorno 09 $9.7, arrivata da una pagina.
```

9.2 La tabella giusta: find("table") prende la prima

```
percorso = Path("dati") / "logisud" / "tariffe.html"

testo_html = percorso.read_text(encoding="utf-8")

pagina = BeautifulSoup(testo_html, "html.parser")

print("Tabelle nella pagina:", len(pagina.find_all("table")))

# find() prende la PRIMA in ordine di documento: i supplementi, non le zone.

prima = pagina.find("table")

print(f"Prima tabella: classe {prima.get('class')}, {len(prima.find_all('tr'))} righe")

print(f"  prima cella: {prima.find('td').get_text(strip=True)}")

# Per id: unico nella pagina (§3.3), dice QUALE tabella volete.

tariffe = pagina.select_one("#tariffe")

print(f"#tariffe: id {tariffe.get('id')}, {len(tariffe.find_all('tr'))} righe")

# Dentro la tabella scelta, solo il corpo: niente intestazione, niente piede.

print(f"  righe del corpo: {len(tariffe.select('tbody tr'))}")


# TRABOCCHETTO: select("tbody tr") sulla PRIMA tabella da' zero righe, senza
#  errori: i supplementi nel sorgente non hanno <tbody>, e html.parser non
#  lo aggiunge (§4.4). Il for successivo fa zero giri, il listino esce vuoto.


# Tabelle nella pagina: 2

# Prima tabella: classe ['supplementi'], 4 righe

#  prima cella: Consegna al piano

# #tariffe: id tariffe, 7 righe

#  righe del corpo: 5
```

9.3 Dalla tabella al dizionario: il listino zona -> prezzo

```
percorso = Path("dati") / "logisud" / "tariffe.html"

pagina = BeautifulSoup(percorso.read_text(encoding="utf-8"), "html.parser")

listino = {}

su_preventivo = []

for tr in pagina.select("#tariffe tbody tr"):

    # Tre celle, tre nomi: l'unpacking funziona anche su una lista di tag.
    cella_zona, cella_prezzo, cella_consegna = tr.find_all("td")

    zona = cella_zona.get_text(strip=True)

    try:

        listino[zona] = prezzo_da_testo(cella_prezzo.get_text(strip=True))

    except ValueError:

        su_preventivo.append(zona)

print("Listino:", listino)

print("Su preventivo:", su_preventivo)

# Il listino che 07.4 scriveva a mano, copiato dai suoi DATI DI PARTENZA.
LISTINO_INIZIALE = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}

print("Uguale a quello di 07.4?", listino == LISTINO_INIZIALE)

# TRABOCCHETTO: listino["ESTERO"] adesso da' KeyError: 'ESTERO' (Giorno 07
# $10.3), perche' la zona e' finita nell'altra lista. Chi chiede un
# preventivo per l'estero va controllato con "in" o con .get() PRIMA.

# Listino: {'NORD': 8.5, 'CENTRO': 7.0, 'SUD': 9.5, 'ISOLE': 14.0}

# Su preventivo: ['ESTERO']

# Uguale a quello di 07.4? True
```

9.4 I link: relativi, assoluti, ancore, mailto:

```
def forma_del_link(href):  
    """Restituisce il nome della forma di un href, anche quando manca."""  
  
    if href is None:  
        return "senza href"  
  
    if href.startswith("#"):  
        return "ancora"  
  
    if href.startswith("mailto:"):  
        return "mailto"  
  
    if href.startswith("https://") or href.startswith("http://"):  
        return "assoluto"  
  
    if href.startswith("/"):  
        return "relativo alla radice"  
  
    return "relativo alla pagina"  
  
# HTML: sette <a> dell'albo di Villanova, uno per forma, e uno senza href.  
pagina = BeautifulSoup(HTML, "html.parser")  
for link in pagina.find_all("a"):  
    # .get() e non ["href"]: il link disattivo non ce l'ha (§7.3).  
    href = link.get("href")  
  
    print(f"{forma_del_link(href):<22}{href}")  
  
# TRABOCCHETTO: l'ordine dei test e' quello delle soglie del Giorno 04. Un  
# href come "//www.regione.example/" comincia con "/" e finisce fra i  
# relativi alla radice, ma e' un sito esterno: la doppia barra vuol dire  
# "stesso schema, altro sito". Per questo il §9.6 non si fida del prefisso.
```

9.5 urljoin(): dal link relativo all'indirizzo completo

```
from urllib.parse import urljoin

# L'indirizzo della pagina da cui avete letto i link: e' la "cartella
# corrente" da cui si leggono gli href relativi.
URL_PAGINA = "https://www.comune.villanova.example/albo/albo.html"

HREF = ["atti/2026-0142.pdf", "/servizi/", "../bandi/mensa-scolastica.html",
        "https://www.regione.example/", "#avvisi"]

for href in HREF:
    # Primo argomento la BASE, secondo il link: l'ordine conta.
    print(urljoin(URL_PAGINA, href))

# TRABOCCHETTO: URL_PAGINA + href incolla le stringhe e basta:
#   ".../albo/albo.htmlatti/2026-0142.pdf", un indirizzo che non esiste. E
#   con gli argomenti invertiti, urljoin(href, URL_PAGINA), le cinque righe
#   sono tutte uguali: la base e' gia' assoluta, e vince lei.

# https://www.comune.villanova.example/albo/atti/2026-0142.pdf
# https://www.comune.villanova.example/servizi/
# https://www.comune.villanova.example/bandi/mensa-scolastica.html
# https://www.regione.example/
# https://www.comune.villanova.example/albo/albo.html#avvisi
```

9.6 Interni ed esterni: urlparse().netloc e il dominio

```
URL_PAGINA = "https://www.comune.villanova.example/albo/albo.html"
DOMINIO = urlparse(URL_PAGINA).netloc

conteggi = {}
per_dominio = {}
con_http = 0

for link in pagina.find_all("a"):
    href = link.get("href")

    # Prima si tolgono le forme che non portano a un'altra pagina...
    if href is None:
        forma = "senza href"

    elif href.startswith("#"):
        forma = "ancora"

    elif href.startswith("mailto:"):
        forma = "mailto"

    else:
        # ...poi si risolve l'indirizzo ($9.5) e si confronta il dominio.
        if href.startswith("http"):
            con_http += 1

            dominio = urlparse(urljoin(URL_PAGINA, href)).netloc

            if dominio == DOMINIO:
                forma = "interno"
            else:
                forma = "esterno"

            per_dominio[dominio] = per_dominio.get(dominio, 0) + 1

# Il conta-occorrenze del Gruppo 07 $12.2, sulle forme dei link.
conteggi[forma] = conteggi.get(forma, 0) + 1
```


9.6 Quattro http, tre esterni: assoluto non vuol dire esterno

```
print("Tag <a>:", sum(conteggi.values()))

for forma, quanti in conteggi.items():

    print(f"  {forma:<11}{quanti:>3}")

print("Cominciano con http:", con_http, "ma gli esterni sono", conteggi["esterno"])

# La barra di cancelletti del Giorno 07 $12.3, un # per link.

for dominio, quanti in per_dominio.items():

    print(f"  {dominio:<36}{'#' * quanti} {quanti}")


# TRABOCCHETTO: togliete il ramo dell'ancora. "#avvisi" passa da urljoin,
# diventa ".../albo/albo.html#avvisi", ha il dominio del Comune: gli
# interni salgono da 8 a 10. Togliete quello del mailto: urljoin lo lascia
# com'e', il suo netloc e' una stringa vuota, e fra gli esterni compare un
# dominio senza nome.


# Tag <a>: 15

#   interno      8
#   ancora       2
#   esterno      3
#   senza href   1
#   mailto       1

# Cominciano con http: 4 ma gli esterni sono 3

#   www.regione.example          ## 2
#   www.gazzettaamministrativa.example # 1
```

10. PIÙ PAGINE: LA PAGINAZIONE

Leggere, cercare il link, seguirlo, fermarsi quando non c'è più

10.1 Il link «Successiva»: il while che si ferma sul None

```
CARTELLA = Path("dati") / "novastore" / "catalogo"

nome_file = "pagina_1.html"          # la prima lettura, PRIMA del ciclo
numero = 0

schede_totali = 0

while nome_file is not None:

    numero += 1

    testo_html = (CARTELLA / nome_file).read_text(encoding="utf-8")

    pagina = BeautifulSoup(testo_html, "html.parser")

    schede = pagina.find_all("li", class_="prodotto")

    schede_totali += len(schede)

    print(f"{numero}. {nome_file}: {len(schede)} schede")

    # L'aggiornamento IN CODA al corpo: la pagina dopo, o None se non c'è.
    successiva = pagina.find("a", class_="successiva")

    if successiva is None:

        nome_file = None

    else:

        nome_file = successiva["href"]

print(f"Pagine lette: {numero}, schede: {schede_totali}")

# TRABOCCHETTO: scrivete l'aggiornamento senza l'if, nome_file =
# successiva["href"]. Sulle prime due pagine funziona; sulla terza
# successiva è None e il programma si ferma con TypeError: 'NoneType'
# object is not subscriptable. È il None del Giorno 06 $6.1, arrivato
# da find() invece che da una funzione senza return.
```

10.2 leggi_pagina(url): il solo punto fra disco e rete

```
URL_CATALOGO = "https://www.novastore.example/catalogo/pagina_1.html"

CARTELLA_SITO = Path("dati") / "novastore"

def leggi_pagina(url):
    """Restituisce il testo HTML della pagina all'indirizzo url.

    Versione su disco: il percorso dell'indirizzo diventa un percorso sotto
    dati/novastore/. E' l'unica funzione del programma che sa da dove arriva
    la pagina: per passare alla rete si cambia lei, e nient'altro ($11.3).
    """
    percorso_url = urlparse(url).path

    # TRABOCCHETTO: la barra iniziale. CARTELLA_SITO / "/catalogo/..." butta
    # via dati/novastore e riparte dalla radice del disco (Giorno 08 $7.3):
    # FileNotFoundError su un percorso che non avete mai scritto.

    percorso = CARTELLA_SITO / percorso_url.lstrip("/")

    return percorso.read_text(encoding="utf-8")

def url_successiva(pagina, url):
    """L'indirizzo completo della pagina dopo, oppure None se e' l'ultima."""
    link = pagina.find("a", class_="successiva")

    if link is None:
        return None

    # L'href e' relativo alla pagina corrente: urljoin lo completa ($9.5).

    return urljoin(url, link["href"])
```

10.2 Il ciclo per livelli: tre righe, e nessuna parla di file

```
# Il ciclo del §10.1, scritto per livelli (Giorno 06 §7.2): tre righe.
```

```
url = URL_CATALOGO
```

```
while url is not None:
```

```
    pagina = BeautifulSoup(leggi_pagina(url), "html.parser")
```

```
    print(f"{urlparse(url).path:<26}{pagina.title.get_text(strip=True)}")
```

```
    url = url_successiva(pagina, url)
```

```
# /catalogo/pagina_1.html    NovaStore - Catalogo, pagina 1 di 3
```

```
# /catalogo/pagina_2.html    NovaStore - Catalogo, pagina 2 di 3
```

```
# /catalogo/pagina_3.html    NovaStore - Catalogo, pagina 3 di 3
```

Leggere, stampare, passare alla pagina dopo. E' la composizione del Giorno 06 §7.1: main() fa il regista, le funzioni fanno il lavoro. url_successiva() restituisce None apposta, ed e' dichiarato nella docstring.

10.3 Un tetto alle pagine: il sito che gira in tondo

```
MAX_PAGINE = 5

# SITO e leggi_pagina(): in memoria, e la pagina 2 rimanda a se stessa.

url = "https://www.novastore.example/catalogo/pagina_1.html"

lette = 0

# Due condizioni: c'e' una pagina dopo, E non si e' ancora al tetto.

while url is not None and lette < MAX_PAGINE:

    pagina = BeautifulSoup(leggi_pagina(url), "html.parser")

    lette += 1

    print(f"{lette}. {url.split('/')[1]}")

    link = pagina.find("a", class_="successiva")

    if link is None:

        url = None

    else:

        url = urljoin(url, link["href"])

# Se url non e' None, il ciclo e' uscito per il tetto e non per la fine.

if url is not None:

    print(f"[!] fermato al tetto di {MAX_PAGINE} pagine: il sito gira in tondo")

# TRABOCCHETTO: lette <= MAX_PAGINE legge SEI pagine con un tetto di cinque:

# 1'off-by-one del Giorno 05 $14.4, nessun errore e un giro di troppo.

# 1. pagina_1.html

# 2. pagina_2.html

# 3. pagina_2.html

# 4. pagina_2.html

# 5. pagina_2.html

# [!] fermato al tetto di 5 pagine: il sito gira in tondo
```

10.4 La pausa fra una pagina e l'altra: time.sleep()

```
PAUSA_SECONDI = 0.2          # piccola in aula; in rete, il Crawl-delay ($12.2)

CARTELLA = Path("dati") / "novastore" / "catalogo"

nome_file = "pagina_1.html"

lette = 0

pause = 0

while nome_file is not None:

    testo_html = (CARTELLA / nome_file).read_text(encoding="utf-8")

    pagina = BeautifulSoup(testo_html, "html.parser")

    lette += 1

    print(f"letta {nome_file}")

    successiva = pagina.find("a", class_="successiva")

    if successiva is None:

        nome_file = None

    else:

        nome_file = successiva["href"]

        # La pausa sta QUI: solo quando c'e' davvero un'altra pagina da chiedere.

        print(f"  pausa di {PAUSA_SECONDI} secondi")

        time.sleep(PAUSA_SECONDI)

        pause += 1

print(f"Pagine: {lette}, pause: {pause}, attesa: {pause * PAUSA_SECONDI:.1f} s")


# TRABOCCHETTO: time.sleep() come prima riga del corpo sembra equivalente, e
# aggiunge una pausa inutile PRIMA della prima pagina. In coda al corpo,
# fuori dall'else, ne aggiunge una DOPO l'ultima. Tre pagine, due pause.
```

11. SCARICARE DAVVERO: requests

L'unico capitolo che esce in rete: fra disco e rete cambia una funzione

11.1 requests.get(url, timeout=10): la richiesta e la risposta

```
import requests

from bs4 import BeautifulSoup

URL = "https://example.com/"

# timeout=10: dopo dieci secondi senza risposta si rinuncia, con un'eccezione.
# Senza timeout, un server che non risponde blocca il programma per sempre.
risposta = requests.get(URL, timeout=10)
print("Codice di stato:", risposta.status_code)
print("Caratteri ricevuti:", len(risposta.text))

# Da qui in poi e' tutto come su disco: il testo passa al parser.
pagina = BeautifulSoup(risposta.text, "html.parser")
print("Titolo:", pagina.title.get_text(strip=True))

# TRABOCCHETTO: accenti storpiati nel testo ricevuto. requests deduce la
# codifica dalle intestazioni del server, e se il server non la dichiara
# puo' sbagliare: il Giorno 08 $2.4, via rete. Rimedio: la si dichiara con
# risposta.encoding = "utf-8" PRIMA di leggere risposta.text.
```

11.2 Il codice di stato: 200, 404, 403, 429, raise_for_status()

```
import requests

INDIRIZZI = [
    "https://example.com/",
    "https://example.com/una-pagina-che-non-esiste.html",
]

for url in INDIRIZZI:
    risposta = requests.get(url, timeout=10)

    try:
        # Con 200 non fa niente; con 404, 403, 429... solleva HTTPError.
        risposta.raise_for_status()
    except requests.HTTPError as e:
        # Il messaggio di e contiene codice e indirizzo: si mostra il codice.
        print(f"[!] {risposta.status_code}: pagina saltata ({type(e).__name__})")
        continue

    print(f"[OK] {risposta.status_code}: {len(risposta.text)} caratteri")

# TRABOCCHETTO: senza raise_for_status(), la pagina d'errore passa al parser
# come una pagina qualsiasi. find_all() restituisce una lista vuota, il
# ciclo fa zero giri, e il report dice "0 prodotti": sembra un catalogo
# vuoto, ed e' un indirizzo sbagliato.
```

11.3 User-Agent e leggi_pagina() di rete: una funzione sola

```
AGENTE = "CorsoScraperBot/1.0"

def leggi_pagina(url):
    """Restituisce il testo HTML della pagina all'indirizzo url."""
    # headers: un dizionario (Giorno 07 §10.1) con le intestazioni.
    risposta = requests.get(url, headers={"User-Agent": AGENTE}, timeout=10)
    risposta.raise_for_status()
    return risposta.text

# Il chiamante cattura la FAMIGLIA di eccezioni di requests (Giorno 09 §7.1):
# rete assente, tempo scaduto, codice di errore.
try:
    testo_html = leggi_pagina("https://example.com/")
except requests.RequestException as e:
    print(f"[ERRORE] pagina non letta: {type(e).__name__}")
else:
    print("Caratteri ricevuti:", len(testo_html))

# TRABOCCHETTO: provate con l'indirizzo del catalogo NovaStore. Il dominio
# .example non esiste, e l'eccezione non e' un 404: e' una ConnectionError,
# il server non e' stato trovato. Il ramo except la raccoglie lo stesso,
# perche' e' della stessa famiglia.
```

12. PRIMA DI LANCIARE UNO SCRAPER

Controlli che fa un programma, domande che si pone una persona

12.1 robots.txt: User-agent, Disallow, Allow, Crawl-delay

```
percorso = Path("dati") / "novastore" / "robots.txt"

with open(percorso, encoding="utf-8") as f:

    for riga in f:

        riga = riga.strip()

        if riga == "" or riga.startswith("#"):

            continue

        # split(":", 1): al massimo UN taglio, anche se il valore ha dei ":".

        campo, valore = riga.split(":", 1)

        if campo == "User-agent":

            print(f"Regole per: {valore.strip()}")

        else:

            print(f"  {campo:<12}{valore.strip()}")

# TRABOCCHETTO: con split(":") senza il secondo argomento, una riga come
# "Sitemap: https://www.novastore.example/sitemap.xml" si spezza in tre
# pezzi (c'e' anche il ":" di https) e l'unpacking si ferma con
# ValueError: too many values to unpack (expected 2, got 3).

# Regole per: RaccoltaPrezziBot

# Disallow /

# Regole per: *

# Disallow /catalogo/riservati/

# Allow /catalogo/

# Disallow /carrello/

# Disallow /account/

# Disallow /cerca

# Crawl-delay 2
```

12.2 urllib.robotparser: can_fetch() e crawl_delay()

```
AGENTE = "CorsoScraperBot/1.0"

SITO = "https://www.novastore.example"

INDIRIZZI = ["/catalogo/pagina_1.html", "/catalogo/riservati/offerte-iscritti.html",
              "/carrello/", "/account/accedi.html", "/chi-siamo.html",
              "/cerca?q=cuffie"]

controllore = RobotFileParser()

# parse() vuole una LISTA di righe: splitlines() la ricava (Giorno 08 $5.5).
testo = (Path("dati") / "novastore" / "robots.txt").read_text(encoding="utf-8")
controllore.parse(testo.splitlines())

for indirizzo in INDIRIZZI:
    # can_fetch vuole l'indirizzo COMPLETO, sito compreso.
    print(f"{indirizzo:<44}{controllore.can_fetch(AGENTE, SITO + indirizzo)}")

print("Crawl-delay per", AGENTE, "->", controllore.crawl_delay(AGENTE))

altro = "RaccoltaPrezziBot/2.0"

print(altro, "->", controllore.can_fetch(altro, SITO + "/catalogo/pagina_1.html"),
      controllore.crawl_delay(altro))

# TRABOCCHETTO: /chi-siamo.html risponde True, ma nel file non c'e' nessun
# Allow che lo nomini. Nessuna regola combacia, e quando nessuna regola
# combacia la risposta e' "consentito". Il robots.txt elenca i divieti,
# non i permessi.
```

12.2 Che cosa risponde robotparser, indirizzo per indirizzo

/catalogo/pagina_1.html	True
/catalogo/riservati/offerte-iscritti.html	False
/carrello/	False
/account/accedi.html	False
/chi-siamo.html	True
/cerca?q=cuffie	False
Crawl-delay per CorsoScraperBot/1.0 ->	2
RaccoltaPrezziBot/2.0 ->	False None

Per voi 2 secondi: il PAUSA_SECONDS in rete. /cerca vieta tutto cio' che comincia con /cerca. CHICCA: per robotparser vince la prima regola che combacia, non la piu' specifica. Disallow: /catalogo/riservati/ va PRIMA di Allow: /catalogo/, o le riservate diventano visitabili.

12.3 Termini d'uso e carico sul server: che cosa si controlla

```
CRAWL_DELAY = 2          # secondi, dal robots.txt di NovaStore (§12.2)

for pagine in [3, 300, 5000]:
    # Fra N pagine ci sono N - 1 pause, come nel §10.4.
    pause = pagine - 1
    secondi = pause * CRAWL_DELAY

    # Divisione intera e resto: i due operatori del Giorno 02 §6.2.
    minuti = secondi // 60
    resto = secondi % 60

    print(f"{pagine:>5} pagine: {pause:>5} pause, {minuti:>3} min {resto:02d} s")

# TRABOCCHETTO: "pagine * CRAWL_DELAY" conta una pausa di troppo. Su 5000
#  pagine l'errore e' di due secondi e non si vede; su 3 pagine la stima
#  dice 6 secondi invece di 4, un terzo in piu'. Fra N pagine, N - 1 pause.

#    3 pagine:      2 pause,    0 min 04 s
#   300 pagine:    299 pause,    9 min 58 s
#  5000 pagine:   4999 pause, 166 min 38 s
```


12.4 I dati personali: raccogliere solo i campi che servono

```
HTML = """
<section class="recensioni" data-codice="NS-2001">

  <div class="recensione" data-autore="utente_4821" data-voto="5">

    <p>Ottimo, la batteria dura.</p></div>

  <div class="recensione" data-autore="utente_0377" data-voto="3">

    <p>Buone, ma stringono.</p></div>

  <div class="recensione" data-autore="utente_1290" data-voto="4">

    <p>Il microfono gracchia un po'.</p></div>

</section>
"""

pagina = BeautifulSoup(HTML, "html.parser")
sezione = pagina.find("section", class_="recensioni")
voti = []

for recensione in sezione.find_all("div", class_="recensione"):

    # Si estrae SOLO il voto: e' l'unico campo che serve allo scopo.
    voti.append(int(recensione["data-voto"]))

record = {"codice": sezione["data-codice"], "recensioni": len(voti),
          "voto_medio": round(sum(voti) / len(voti), 2)}

print(record)

# TRABOCCHETTO: salvare recensione.get_text() "per completezza" porta dentro
#   il testo libero, e un testo libero puo' contenere di tutto: un nome, un
#   telefono, un indirizzo. La minimizzazione si fa sui campi, non a valle.

# {'codice': 'NS-2001', 'recensioni': 3, 'voto_medio': 4.0}
```

12.5 La lista di controllo: sette domande prima di premere Esegui

#	Domanda	Come si verifica	Se la risposta è no
1	Il robots.txt consente gli indirizzi da visitare?	can_fetch() su ogni indirizzo (§12.2)	quegli indirizzi non si visitano
2	I termini d'uso non vietano la raccolta automatica?	si leggono, prima (§12.3)	ci si ferma e si chiede a chi ne risponde
3	Raccogliete solo i campi che servono?	l'elenco delle chiavi del record (§8.4)	si tolgono i campi in più
4	Nessun dato personale, o una decisione di chi ne risponde?	la pagina, campo per campo (§12.4)	non si estraggono
5	Pausa e tetto alle pagine sono impostati?	PAUSA_SECONDS, MAX_PAGINE in cima	si aggiungono prima di partire
6	L'agente è dichiarato con un nome riconoscibile?	AGENTE nelle intestazioni (§11.3)	si dichiara: niente nomi di browser
7	I dati sono davvero nell'HTML della pagina?	«quanti record» dà più di zero (§8.5)	si cerca la sorgente dei dati (§13.2)

Con la sesta risposta a False, il programma della slide seguente stampa [!] Fermi: 1 controllo non superato su 7, e sotto la domanda che manca.

12.5 controlli_non_superati(): il motivo viaggia con il rifiuto

```
CONTROLLI = [
    ("robots.txt consente gli indirizzi?", True),
    ("I termini d'uso non vietano la raccolta automatica?", True),
    ("Solo i campi che servono?", True),
    ("Nessun dato personale, o deciso da chi ne risponde?", True),
    ("Pausa e tetto alle pagine impostati?", True),
    ("Agente dichiarato con un nome riconoscibile?", False),
    ("I dati sono nell'HTML della pagina?", True),
]

def controlli_non_superati(controlli):
    mancanti = []

    for domanda, risposta in controlli:
        if not risposta:
            mancanti.append(domanda)

    return mancanti

mancanti = controlli_non_superati(CONTROLLI)

if mancanti:
    print(f"[!] Fermi: {len(mancanti)} controllo non superato su {len(CONTROLLI)}")

    for domanda in mancanti:
        print("    -", domanda)
else:
    print("[OK] Si può lanciare")

# TRABOCCHETTO: una funzione che restituisse solo True o False direbbe "no"

# senza dire perche'. Restituire la lista delle domande fallite e' la

# stessa scelta del $8.2: il motivo viaggia insieme al rifiuto.
```

13. LE PAGINE DINAMICHE: IL LIMITE DI QUESTA TECNICA

Il browser vede cinque righe, il programma zero: e hanno ragione tutti e due

13.1 La tabella vuota: il contenuto che arriva con JavaScript

```
percorso = Path("dati") / "aurora" / "orari.html"

pagina = BeautifulSoup(percorso.read_text(encoding="utf-8"), "html.parser")

# Il controllo "quanti record" del §8.5: qui da' zero, ed e' il sintomo.
righe = pagina.select("#orari tbody tr")

print("Righe nella tabella:", len(righe))

# L'intestazione invece c'e': la tabella esiste, e' vuota.
print("Colonne dell'intestazione:", len(pagina.select("#orari thead th")))

# Due indizi della pagina dinamica: un messaggio d'attesa e uno script.
print("Testo di stato:", pagina.select_one("#stato").get_text(strip=True))
print("Script nella pagina:", len(pagina.find_all("script")))

# TRABOCCHETTO: il for sulle righe fa zero giri e il report dice "0 servizi",
#   senza errori, come il secondo for sul file gia' letto del Giorno 08 §4.5.
#   Quando un conteggio dice zero su una pagina che nel browser e' piena, la
#   prima ipotesi da verificare e' questa, non il selettore.

# Righe nella tabella: 0
# Colonne dell'intestazione: 4
# Testo di stato: Caricamento degli orari in corso...
# Script nella pagina: 1
```

13.2 Il dato alla fonte: l'indirizzo JSON dichiarato nella pagina

```
URL_ORARI = "https://www.aurora.example/orari.html"

pagina = BeautifulSoup(leggi_pagina(URL_ORARI), "html.parser")

# La pagina dichiara la fonte in un attributo data-* (§3.3).
sorgente = pagina.select_one("#orari")["data-sorgente"]

url_json = urljoin(URL_ORARI, sorgente)

print("Sorgente dichiarata:", sorgente)

print("Indirizzo completo: ", url_json)

dati = json.loads(leggi_pagina(url_json))

print("Servizi:", len(dati["orari"]))

for voce in dati["orari"]:

    orario = f"{voce['apertura']}-{voce['chiusura']}"

    print(f"  {voce['servizio']}<24}{voce['giorni']<9}{orario}")

# TRABOCCHETTO: dati["orari"] e non dati[0]. Il JSON e' un DIZIONARIO con
#   dentro la lista; dati[0] cerca la chiave 0 e si ferma con KeyError: 0.
#   Aprite il file e guardate il primo carattere: { e' dizionario, [ e' lista.

# Sorgente dichiarata: api/orari.json

# Indirizzo completo: https://www.aurora.example/api/orari.json

# Servizi: 5

#   Punto prelievi          lun-sab  07:30-10:30
#   Sportello prenotazioni  lun-ven  08:00-18:00
#   Ritiro referti         lun-ven  11:00-13:00
#   Cardiologia            mar-gio  14:00-19:00
#   Radiologia             lun-ven  08:30-16:30
```

14. SALVARE I DATI ESTRATTI

JSON per un programma, CSV per una persona: sempre in output/, mai in dati/

14.1 JSON: la lista di dizionari così com'è, ensure_ascii=False

```
record = [

    {"codice": "NS-2009", "nome": "Cuffie Bass & Treble",

    "categoria": "AUDIO", "prezzo": 89.0, "disponibile": True},

    {"codice": "NS-2012", "nome": "Lampada da scrivania Novità",

    "categoria": "ACCESSORI", "prezzo": 27.0, "disponibile": True},

]

# La cartella output/ si crea da sola se manca (Giorno 08 §8.3).

cartella = Path("output")

cartella.mkdir(exist_ok=True)

percorso = cartella / "catalogo.json"

with open(percorso, "w", encoding="utf-8") as f:

    # indent=2 per chi lo apre a occhio; ensure_ascii=False per la "à".

    json.dump(record, f, indent=2, ensure_ascii=False)

print(percorso.read_text(encoding="utf-8"))

# La differenza, su una parola sola.

print(json.dumps("Novità"))

print(json.dumps("Novità", ensure_ascii=False))

# TRABOCCHETTO: ensure_ascii=False senza encoding="utf-8" nella open() e'

# la combinazione peggiore: su un Windows con la codifica di sistema

# diversa, la "à" si scrive in un'altra codifica e il file riletto con

# utf-8 si ferma con UnicodeDecodeError (Giorno 08 §2.4). Vanno insieme.
```


14.1 Il file salvato, e la «à» con e senza ensure_ascii

```
[
  {
    "codice": "NS-2009",
    "nome": "Cuffie Bass & Treble",
    "categoria": "AUDIO",
    "prezzo": 89.0,
    "disponibile": true
  },
  {
    "codice": "NS-2012",
    "nome": "Lampada da scrivania Novità",
    "categoria": "ACCESSORI",
    "prezzo": 27.0,
    "disponibile": true
  }
]
"Novit\u00e0"
"Novità"
```

La & arrivata come & nel file e' una & qualsiasi: di HTML non resta traccia. True e' diventato true, e tornera' True alla rilettura (Giorno 08 §12.3). Le ultime due righe: la stessa parola, con e senza ensure_ascii=False.

14.2 CSV con csv.DictWriter: una colonna per ogni chiave

```
# Le cinque chiavi del record (§8.4): sono anche le colonne, in quest'ordine.
CAMPI = ["codice", "nome", "categoria", "prezzo", "disponibile"]

record = [

    {"codice": "NS-2001", "nome": "Cuffie wireless", "categoria": "AUDIO",
     "prezzo": 49.9, "disponibile": True},

    {"codice": "NS-2005", "nome": "Casse bluetooth", "categoria": "AUDIO",
     "prezzo": 59.0, "disponibile": False},

    {"codice": "NS-2009", "nome": "Cuffie Bass & Treble", "categoria": "AUDIO",
     "prezzo": 89.0, "disponibile": True},

]

cartella = Path("output")
cartella.mkdir(exist_ok=True)
percorso = cartella / "catalogo.csv"

# newline="": il fine riga lo mette il modulo csv, non la open().

with open(percorso, "w", encoding="utf-8", newline="") as f:

    # Il ; come separatore, come nei file del Giorno 08 e del Giorno 09.

    scrittore = csv.DictWriter(f, fieldnames=CAMPI, delimiter=";")

    scrittore.writeheader()

    scrittore.writerows(record)

print(percorso.read_text(encoding="utf-8"))

# TRABOCCHETTO: senza newline="" nella open(), su Windows ogni riga del CSV
# e' seguita da una riga vuota: il modulo scrive il suo fine riga e la
# open() ne aggiunge un altro. Sul Mac e su Linux non si vede, ed e' per
# questo che il difetto arriva al collega e non a voi.
```

14.3 Rileggere e verificare: l'assert sul numero di record

```
# record: NS-2001 e NS-2005 come al §14.2, salvati in output/ in JSON e in CSV.

# Rilettura: json.load() restituisce la lista di dizionari...

with open(cartella / "catalogo.json", encoding="utf-8") as f:

    da_json = json.load(f)

# ...DictReader una riga alla volta, ognuna come dizionario con le chiavi

# prese dall'intestazione del file.

da_csv = []

with open(cartella / "catalogo.csv", encoding="utf-8", newline="") as f:

    for riga in csv.DictReader(f, delimiter=";"):

        da_csv.append(riga)


assert len(da_json) == len(record), "il JSON ha perso dei record"

assert len(da_csv) == len(record), "il CSV ha perso dei record"

print(f"[OK] {len(record)} salvati, {len(da_json)} rilette dal JSON, "

      f"{len(da_csv)} dal CSV")


# [OK] 2 salvati, 2 rilette dal JSON, 2 dal CSV
```

14.3 Stesso record, due viaggi: guardate i tipi

```
# Stesso record, due viaggi: guardate i TIPI, non i valori stampati.

voce_json = da_json[1]

voce_csv = da_csv[1]

print("JSON:", voce_json["prezzo"], type(voce_json["prezzo"]).__name__,
      voce_json["disponibile"])

print("CSV: ", voce_csv["prezzo"], type(voce_csv["prezzo"]).__name__,
      voce_csv["disponibile"])

print("Uguali all'originale?", da_json == record, da_csv == record)

print("bool('False') =", bool(voce_csv["disponibile"]))

# TRABOCCHETTO: dal CSV "disponibile" torna come la STRINGA "False", e una
#   stringa non vuota in un if e' vera. Il programma che rilegge il CSV e fa
#   if voce["disponibile"]: mostra come disponibili le casse esaurite. Si
#   converte: disponibile = voce["disponibile"] == "True".

# JSON: 59.0 float False
# CSV:  59.0 str False
# Uguali all'originale? True False
# bool('False') = True
```

14.4 Lo scraper completo: scarica, estrai, pulisci, salva

```
AGENTE = "CorsoScraperBot/1.0"

URL_CATALOGO = "https://www.novastore.example/catalogo/pagina_1.html"

URL_ROBOTS = "https://www.novastore.example/robots.txt"

PAUSA_SECONDI = 0.2

MAX_PAGINE = 5

CARTELLA_SITO = Path("dati") / "novastore"

CAMPI = ["codice", "nome", "categoria", "prezzo", "disponibile"]

# leggi_pagina() ($10.2), testo_campo() ($5.5) e prezzo_da_testo() ($8.2):
# le stesse, identiche. Qui si aggiunge soltanto estrai_record().

def estrai_record(scheda):

    """Una scheda diventa un dizionario; ValueError se il prezzo non regge."""

    return {

        "codice": scheda["data-codice"],

        "nome": testo_campo(scheda, "h2", "nome"),

        "categoria": scheda.get("data-categoria"),

        "prezzo": prezzo_da_testo(testo_campo(scheda, "span", "prezzo")),

        "disponibile": "esaurito" not in scheda["class"],

    }
```

14.4 FASE 1 - SCARICA: il permesso prima di qualunque pagina

```
def main():  
    # FASE 1 - SCARICA. Il permesso prima di qualunque pagina.  
  
    controllore = RobotFileParser()  
    controllore.parse(leggi_pagina(URL_ROBOTS).splitlines())  
    print(f"Agente {AGENTE}, Crawl-delay {controllore.crawl_delay(AGENTE)}")  
  
    record = []  
    scarti = {}  
    codici_visti = []  
    lette = 0  
    pagine = 0  
    url = URL_CATALOGO  
    while url is not None and pagine < MAX_PAGINE:  
        if not controllore.can_fetch(AGENTE, url):  
            print(f"[!] vietata da robots.txt: {urlparse(url).path}")  
            break  
  
        pagina = BeautifulSoup(leggi_pagina(url), "html.parser")  
        pagine += 1  
        schede = pagina.find_all("li", class_="prodotto")  
        print(f"Pagina {pagine}: {len(schede)} schede")
```

14.4 FASI 2 E 3 - ESTRAI E PULISCI, una scheda alla volta

```
# FASI 2 E 3 - ESTRAI E PULISCI, una scheda alla volta.

for scheda in schede:

    lette += 1

    codice = scheda["data-codice"]

    if codice in codici_visti:

        scarti["duplicato"] = scarti.get("duplicato", 0) + 1

        continue

    codici_visti.append(codice)

    try:

        record.append(estrai_record(scheda))

    except ValueError as e:

        motivo = str(e).split(":")[0]

        scarti[motivo] = scarti.get(motivo, 0) + 1

link = pagina.find("a", class_="successiva")

if link is None:

    url = None

else:

    url = urljoin(url, link["href"])

    time.sleep(PAUSA_SECONDI)

# TRABOCCHETTO: spostate lette += 1 sotto il controllo dei doppioni. Il
# Mouse ripetuto esce con continue PRIMA di essere contato, lette vale 16,
# e l'assert si ferma con AssertionError: schede perse per strada. E' il
# motivo per cui il contatore delle lette e' la prima riga del for.
```

14.4 L'invariante, poi FASE 4 - SALVA e rileggi

```
scartate = sum(scarti.values())

# L'invariante del Giorno 09 $14.4: nessuna scheda persa per strada.
assert lette == len(record) + scartate, "schede perse per strada"

print(f"Schede lette: {lette} valide: {len(record)} scartate: {scartate}")

for motivo, quanti in scarti.items():

    print(f"    {motivo:<22}{quanti}")

totale = 0.0

for voce in record:

    totale += voce["prezzo"]

print(f"Totale: {totale:.2f} EUR media: {totale / len(record):.2f} EUR")

# FASE 4 - SALVA, e rileggi per verificare.

cartella = Path("output")

cartella.mkdir(exist_ok=True)

percorso = cartella / "catalogo_novastore.csv"

with open(percorso, "w", encoding="utf-8", newline="") as f:

    scrittore = csv.DictWriter(f, fieldnames=CAMPI, delimiter=";")

    scrittore.writeheader()

    scrittore.writerows(record)

riletti = []

with open(percorso, encoding="utf-8", newline="") as f:

    for riga in csv.DictReader(f, delimiter=";"):

        riletti.append(riga)

assert len(riletti) == len(record), "il CSV non ha tutte le righe"

# Si stampa .name, mai il percorso intero: cambia da macchina a macchina.

print(f"[OK] {percorso.name}: {len(riletti)} righe salvate e rilette")
```


14.4 Diciassette schede, tredici prodotti, quattro scarti

```
Agente CorsoScraperBot/1.0, Crawl-delay 2
Pagina 1: 6 schede
Pagina 2: 6 schede
Pagina 3: 5 schede
Schede lette: 17  valide: 13  scartate: 4
    prezzo assente          1
    prezzo non numerico     1
    prezzo vuoto            1
    duplicato                1
Totale: 1849.10 EUR  media: 142.24 EUR
[OK] catalogo_novastore.csv: 13 righe salvate e rilette
```

In coda al file, if `__name__ == "__main__"`: `main()`. Solo `leggi_pagina()` sa che le pagine arrivano dal disco: per andare in rete si sostituisce lei (§11.3), e `PAUSA_SECONDI` prende il valore del `Crawl-delay`.

15. ERRORI TIPICI DELLA GIORNATA

Metà non danno errore: il programma finisce, e i numeri sono sbagliati

```
HTML = """
<li class="prodotto" data-codice="NS-2009"><span class="prezzo">89,00 &euro;</span></li>
<li class="prodotto" data-codice="NS-2010"><h2 class="nome">Docking station</h2></li>
"""

pagina = BeautifulSoup(HTML, "html.parser")

for scheda in pagina.find_all("li", class_="prodotto"):
    prezzo_testo = scheda.find("span", class_="prezzo").get_text(strip=True)
    print(scheda["data-codice"], len(prezzo_testo), "caratteri")

# TRABOCCHETTO: la prima scheda passa, la seconda no. Il traceback indica la
# riga di get_text(), ma il difetto e' nella pagina: NS-2010 non ha lo span.
# Su cento prodotti si ferma al primo senza prezzo, e gli altri si perdono.

# NS-2009 7 caratteri

# Traceback (most recent call last):
# File "prova.py", line 9, in <module>
#     prezzo_testo = scheda.find("span", class_="prezzo").get_text(strip=True)
#
#     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
#
# AttributeError: 'NoneType' object has no attribute 'get text'
```

E2 — Zero risultati senza errori: selettore, <tbody>, pagina dinamica

```
# apri(relativo): l'albero di una pagina di dati/, letta e passata al parser.

catalogo = apri("novastore/catalogo/pagina_1.html")

tariffe = apri("logisud/tariffe.html")

orari = apri("aurora/orari.html")

# Tre cause di zero, ognuna accanto alla sua versione che funziona.

PROVE = [

    ("refuso nella classe", catalogo, "li.prodotti"),

    ("classe giusta", catalogo, "li.prodotto"),

    ("tbody copiato dal browser", tariffe, "table.supplementi > tbody > tr"),

    ("senza tbody", tariffe, "table.supplementi tr"),

    ("pagina dinamica", orari, "#orari tbody tr"),

]

for descrizione, pagina, selettore in PROVE:

    print(f"{descrizione:<27}{len(pagina.select(selettore)):>3}  {selettore}")


# TRABOCCHETTO: nessuna delle tre righe a zero solleva un errore. select()

#  risponde sempre con una lista, vuota se non trova niente: il for che

#  segue fa zero giri e il report esce pulito, con zero prodotti.


# refuso nella classe          0  li.prodotti

# classe giusta                6  li.prodotto

# tbody copiato dal browser    0  table.supplementi > tbody > tr

# senza tbody                  4  table.supplementi tr

# pagina dinamica              0  #orari tbody tr
```

E3 — find_all sulla pagina: i prezzi che scivolano di una riga

```
HTML = """
<aside><span class="prezzo">15,90 &euro;</span></aside>
<li class="prodotto"><h2 class="nome">Webcam HD</h2>
    <span class="prezzo">49,00 &euro;</span></li>
<li class="prodotto"><h2 class="nome">Docking station</h2></li>
<li class="prodotto"><h2 class="nome">Lampada da scrivania</h2>
    <span class="prezzo">27,00 &euro;</span></li>
"""

pagina = BeautifulSoup(HTML, "html.parser")

# Due liste parallele, cercate sulla PAGINA intera (Giorno 07 §6.4).
nomi = pagina.find_all("h2", class_="nome")
prezzi = pagina.find_all("span", class_="prezzo")

print(f"nomi: {len(nomi)}, prezzi: {len(prezzi)}")

for i in range(len(nomi)):

    print(f" {nomi[i].get_text():<22}{prezzi[i].get_text().split()[0]}")

# TRABOCCHETTO: le due liste hanno la stessa lunghezza per caso: un prezzo
#   in piu' (l'offerta del giorno) e uno in meno (la Docking station). Ogni
#   prodotto riceve il prezzo di un altro, e il programma non protesta.

# nomi: 3, prezzi: 3

#   Webcam HD           15,90
#   Docking station     49,00
#   Lampada da scrivania 27,00
```

E4 — class= al posto di class_=: SyntaxError

```
from bs4 import BeautifulSoup

pagina = BeautifulSoup('<li class="prodotto">Webcam HD</li>', "html.parser")
schede = pagina.find_all("li", class="prodotto")
print(len(schede))

# TRABOCCHETTO: class e' una parola riservata di Python (Giorno 02 §3.2),
# e non puo' fare da nome di argomento. Il file non parte nemmeno: nessun
# print viene eseguito, perche' l'errore e' di sintassi e non di esecuzione.

# File "prova.py", line 4
#     schede = pagina.find_all("li", class="prodotto")
#                                     ^^^^^
# SyntaxError: invalid syntax
```

E5 — ModuleNotFoundError: pacchetto, modulo, interprete

```
# Il pacchetto si installa come beautifulsoup4, ma si importa come bs4 (§2.2).
```

```
from beautifulsoup4 import BeautifulSoup
```

```
# TRABOCCHETTO: il nome del pacchetto usato come nome del modulo. Il
```

```
# messaggio e' lo stesso di quando bs4 non e' installato affatto, e porta
```

```
# a reinstallare una libreria che c'e' gia'.
```

```
# Traceback (most recent call last):
```

```
# File "prova.py", line 2, in <module>
```

```
#     from beautifulsoup4 import BeautifulSoup
```

```
# ModuleNotFoundError: No module named 'beautifulsoup4'
```

Si legge il nome fra apici. 'beautifulsoup4': e' sbagliata la riga di import, si scrive from bs4 import BeautifulSoup. 'bs4': la libreria manca per l'interprete che esegue il file. Cura: python -m pip con lo stesso python, e l'interprete del progetto in PyCharm.

```
percorso = Path("dati") / "novastore" / "catalogo" / "pagina_1.html"

pagina = BeautifulSoup(percorso.read_text(encoding="utf-8"), "html.parser")

href = pagina.find("a", class_="successiva")["href"]

print("href letto:", href)

# L'href e' relativo alla PAGINA, e open() lo cerca nella directory di lavoro.

with open(href, encoding="utf-8") as f:

    testo_html = f.read()


# TRABOCCHETTO: il file esiste, dentro dati/novastore/catalogo/. Il messaggio
# dice "No such file or directory" ed e' vero solo dal punto di vista della
# directory di lavoro (Giorno 08 $7.1): la base giusta era la pagina.


# href letto: pagina_2.html


# Traceback (most recent call last):

#   File "prova.py", line 10, in <module>

#       with open(href, encoding="utf-8") as f:

#           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

# FileNotFoundError: [Errno 2] No such file or directory: 'pagina 2.html'
```


E7 — ValueError sul prezzo: "1.249,00", "su richiesta", ""

```
# Tre testi di prezzo presi dal catalogo, convertiti nel modo ingenuo.
for testo in ["1.249,00", "su richiesta", ""]:
    try:
        prezzo = float(testo.replace(",", "", "."))
    except ValueError as e:
        print(f"[!] {e}")

# TRABOCCHETTO: tre errori con lo stesso tipo e lo stesso inizio di
# messaggio, e tre cure diverse: togliere il punto delle migliaia,
# scartare il prodotto, segnalare il campo vuoto. Il tipo raggruppa, non
# spiega (Giorno 09 $9.6).

# [!] could not convert string to float: '1.249.00'
# [!] could not convert string to float: 'su richiesta'
# [!] could not convert string to float: ''
```

E8 — while True senza tetto: il sito che gira in tondo

```
# SITO: il sito difettoso del $10.3, in cui la pagina 2 rimanda a se stessa.

url = "https://www.novastore.example/catalogo/pagina_1.html"

visitati = []          # gli indirizzi gia' letti: 1'operatore in, Giorno 07 $5.1

while url is not None:

    if url in visitati:

        print(f"[!] {url.split('/')[1]} già letta: il sito gira in tondo, stop")

        break

    visitati.append(url)

    pagina = BeautifulSoup(SITO[url], "html.parser")

    print(f"{len(visitati)}. {url.split('/')[1]}")

    link = pagina.find("a", class_="successiva")

    if link is None:

        url = None

    else:

        url = urljoin(url, link["href"])

# TRABOCCHETTO: il confronto e' sull'indirizzo COMPLETO, dopo urljoin. Con

# 1'href nudo, "pagina_2.html" di una cartella e "pagina_2.html" di

# un'altra sembrerebbero la stessa pagina, e il programma si fermerebbe

# su un sito sano.

# 1. pagina_1.html

# 2. pagina_2.html

# [!] pagina_2.html già letta: il sito gira in tondo, stop
```

Riepilogo del giorno

`python -m pip install beautifulsoup4 requests`; e si importa come `bs4`.

HTML: tag e attributi come un dizionario. id unico, class multipla, href, data-.*.

Le entità le traduce il parser: ` ` diventa `\xa0`, che `strip()` toglie e `split(" ")` no.

`BeautifulSoup(testo_html, "html.parser")` costruisce l'albero: `.parent`, `.name`, `.children`.

`find()` dà il primo elemento o `None`, `find_all()` una lista, vuota se non trova niente.

I campi si cercano DENTRO la scheda. `testo_campo()` controlla il `None` con `is None`.

`select()`: punto per la classe, cancelletto per l'id, spazio discendente, `>` figlio.

`get_text(" ", strip=True)`. `tag["href"]` rigido, `tag.get("href")` tollerante. class è lista.

`prezzo_da_testo()`: via €, via i punti, virgola in punto; altrimenti `ValueError` e motivo.

Tabelle come liste di tuple, la tabella giusta per id. `urljoin()` completa i link.

Interno o esterno lo dice il dominio. Paginazione: while fino al `None`, tetto e pausa.

`requests.get(url, timeout=10)`, `raise_for_status()`, User-Agent: cambia una funzione.

robots.txt con `can_fetch()` e `crawl_delay()`; sette domande prima di premere Esegui.

Zero righe su una pagina piena: pagina dinamica, cercate la fonte. Salvate e rileggete.

Grazie.

shaping the skills of tomorrow

